

Appunti del Laboratorio di Ottica

Mirolo Manuele
Michele Tedeschini

Alessio Brusini
Emanuele Stroili

Anno Accademico 2026/2027

Indice

1	Introduzione all'Ottica	1
1.1	Esempi di Formattazione	1
1.2	Info	1

Introduzione all'Ottica

1.1 Esempi di Formattazione

Un esperimento tipico richiede la misura della lunghezza d'onda di un laser He-Ne. Secondo i nostri dati, il valore misurato è $\lambda = (632,8 \pm 0,1) \text{ nm}$.

$$f(x) = \sin\left(\frac{2\pi}{\lambda}x\right) \quad (1.1)$$

Definizione L'ottica geometrica è l'approssimazione in cui la lunghezza d'onda della luce è considerata molto minore delle dimensioni degli oggetti con cui interagisce ($\lambda \rightarrow 0$).

Nota Ricordarsi di allineare sempre gli specchi dell'interferometro di Michelson prima di iniziare la presa dati

Teorema : Legge di Snell

Dati due mezzi con indici di rifrazione n_1 e n_2 , l'angolo di incidenza θ_1 e l'angolo di rifrazione θ_2 sono legati da:

$$n_1 \sin \theta_1 = n_2 \sin \theta_2 \quad (1.2)$$

1.2 Info

8 esperienze relazioni d'esame entro 1 settimana prima dell'appello (le prime relazioni andranno spedite entro 1 settimana) max gruppo da 5 persone (5 gruppi per giorno). 1 solo set di relazioni per gruppo

Introduzione: Analisi Dati con Python (Facoltativo)

In questo capitolo vengono introdotti gli strumenti software per l'analisi statistica dei dati sperimentali. Il framework di riferimento è **Python**, in particolare utilizzando la libreria **iminuit** per l'ottimizzazione dei parametri tramite il metodo dei minimi quadrati (χ^2).

Fit di un Istogramma con una Gaussiana

Il primo esempio mostra come generare un set di dati distribuiti secondo una normale (Gaussiana), riempirne un istogramma e successivamente effettuare un fit per estrarre i parametri della distribuzione.

La funzione Gaussiana utilizzata per il fit è definita come:

$$f(x; C, \mu, \sigma) = C \exp\left(-\frac{1}{2} \left(\frac{x - \mu}{\sigma}\right)^2\right) \quad (1.3)$$

Di seguito il codice completo:

Listing 1.1: Generazione e fit di un istogramma Gaussiano

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from iminuit import Minuit
4
5 # 1. Generazione dati fittizi (Gaussiana con mu=-5.5, sigma=0.5)
6 with open('datpy.txt', 'w') as file:
7     for i in range(1000):
8         g = np.random.normal(-5.5, 0.5)
9         file.write('%s\n' % (g))
10
11 # 2. Lettura dati e creazione istogramma
12 dati = np.loadtxt("datpy.txt")
13 x = dati
14
15 plt.figure()
16 counts, bins, _ = plt.hist(x, range=(-10,0), bins=50, alpha=0.6)
17
18 # 3. Definizione del modello e del Chi Quadrato
19 def gauss(x, par0, par1, par2):
20     return par0 * np.exp(-0.5 * np.power((x - par1) / par2, 2.0))
21
22 def chi2(par0, par1, par2):
23     mask = counts > 0 # Evita la divisione per zero nei bin vuoti
24     # x dei bin valutato nel punto medio
25     x_mid = (bins[:-1] + bins[1:])[mask] / 2
26     y_teo = gauss(x_mid, par0, par1, par2)
27     err_y = np.sqrt(counts[mask])
28     return np.sum(((counts[mask] - y_teo) / err_y)**2)
29
```

```

30 # 4. Minimizzazione con Minuit
31 m = Minuit(chi2, par0=1.0, par1=0.0, par2=1.0)
32 m.migrad() # Algoritmo di minimizzazione
33 m.hesse()  # Calcolo della matrice di covarianza
34 m.minos()  # Stima asimmetrica degli errori (opzionale)
35
36 # Estrazione dei parametri
37 Constant_fit = m.values["par0"]
38 Mean_fit = m.values["par1"]
39 Sigma_fit = m.values["par2"]
40
41 # [...] (Codice per il plot omissso per brevità)
42 plt.show()

```

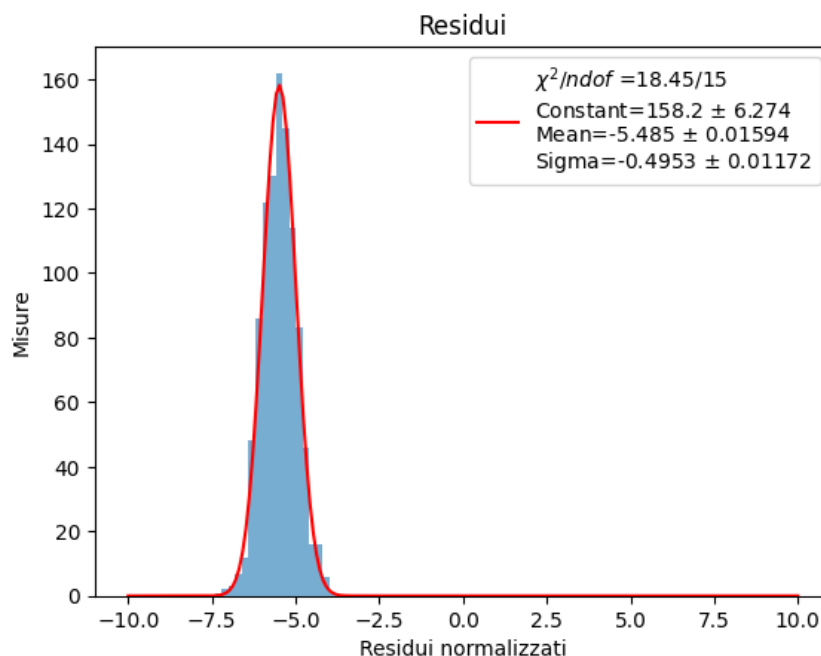


Figura 1.1: Fit di un set di dati generati casualmente secondo una distribuzione Normale ($\mu = -5.5, \sigma = 0.5$). La curva rossa rappresenta la funzione di fit ottimizzata tramite minimizzazione del χ^2 . Il box in alto a destra, generato da `iminuit`, riporta i parametri estratti e i relativi errori.

Fit non lineare, errori su ambo gli assi e Analisi dei Residui

Questo secondo script simula un decadimento esponenziale $y = Ae^{-x/B}$ ed esegue un'analisi completa in tre fasi:

1. **Fit del modello:** Esegue la minimizzazione considerando la varianza efficace.
2. **Distribuzione dei residui:** Verifica che i residui normalizzati, definiti come $r_i = \frac{y_i - f(x_i)}{\sigma_{eff,i}}$, seguano una distribuzione Normale standard $N(0, 1)$.

3. **Andamento dei residui:** Esegue un fit lineare ($y = P_0 + P_1x$) sul grafico dei residui in funzione di x . Se il modello iniziale è corretto, non ci dovrebbero essere trend sistematici (ci si aspetta $P_0 \simeq 0, P_1 \simeq 0$).

Listing 1.2: Fit esponenziale con varianza efficace e studio dei residui

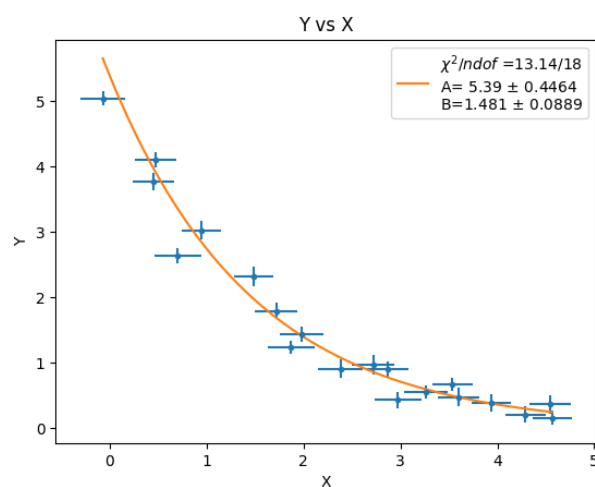
```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from iminuit import Minuit
4
5 # --- 1. LETTURA DATI ---
6 dati = np.loadtxt("datpy.txt")
7 x = dati[:,0]
8 y = dati[:,1]
9 ex = dati[:,2]
10 ey = dati[:,3]
11
12 # --- 2. FIT ESPONENZIALE (METODO VARIANZA EFFICACE) ---
13 def fitfunc(x, par0, par1):
14     return par0 * np.exp(-x / par1)
15
16 def fitfuncprime(x, par0, par1):
17     dx = 1e-6
18     # Derivata numerica
19     return (fitfunc(x+dx/2, par0, par1) - fitfunc(x-dx/2, par0, par1))/dx
20
21 def chi2_eff(par0, par1):
22     # Propagazione dell'errore di x su y tramite derivata
23     sigma_eff = np.sqrt(ey**2 + (ex * fitfuncprime(x, par0, par1))**2)
24     return np.sum(((y - fitfunc(x, par0, par1)) / sigma_eff)**2)
25
26 m1 = Minuit(chi2_eff, par0=1.0, par1=1.0)
27 m1.migrad()
28 A_fit, B_fit = m1.values["par0"], m1.values["par1"]
29
30 # --- 3. ANALISI DISTRIBUZIONE DEI RESIDUI ---
31 y_res = y - fitfunc(x, A_fit, B_fit)
32 ey_eff = np.sqrt(ey**2 + (ex * fitfuncprime(x, A_fit, B_fit))**2)
33
34 counts, bins, _ = plt.hist(y_res / ey_eff, range=(-10,10), bins=20)
35
36 def gauss(x, par0, par1, par2):
37     return par0 * np.exp(-0.5 * np.power((x - par1) / par2, 2.0))
38
39 def chi2_res(par0, par1, par2):
40     mask = counts>0
41     x_mid = (bins[:-1] + bins[1:])[mask]/2
42     return np.sum((((counts[mask] - gauss(x_mid, par0, par1, par2)) / np.sqrt(
43         counts[mask]))**2)
44
45 m2 = Minuit(chi2_res, par0=1.0, par1=0.0, par2=1.0)
46 m2.migrad()
```



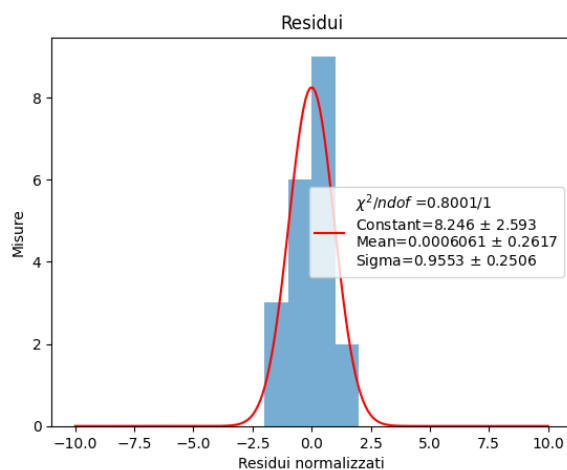
```

46
47 # --- 4. RICERCA TREND SISTEMATICI SUI RESIDUI ---
48 def retta(x, par0, par1):
49     return par0 + par1 * x
50
51 def chi2_trend(par0, par1):
52     return np.sum(((y_res - retta(x, par0, par1)) / ey_eff)**2)
53
54 m3 = Minuit(chi2_trend, par0=0.0, par1=0.0)
55 m3.migrad()

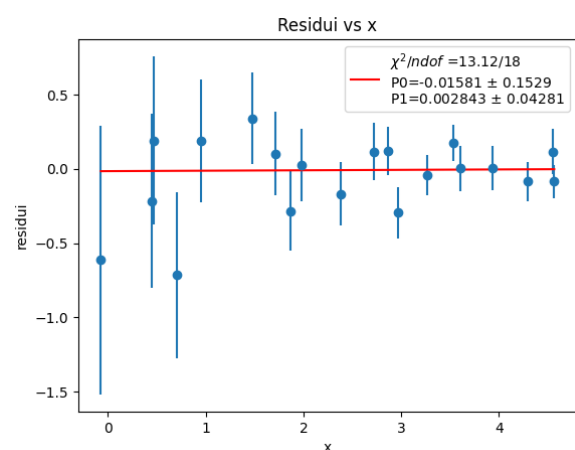
```



(a) Fit del modello esponenziale. Gli errori sulle y includono la varianza efficace derivata dall'incertezza sulle x .



(b) Distribuzione dei residui normalizzati. Il fit Gaussiano conferma che $r_i \sim N(0, 1)$, indicando una corretta stima degli errori.



(c) Andamento dei residui in funzione di x . Il fit lineare non mostra pendenze significative ($P_1 \simeq 0$), escludendo errori sistematici di modello.

Figura 1.2: Analisi completa del fit non lineare ($y = Ae^{-x/B}$) con l'utilizzo della varianza efficace e studio dei residui.

Il Framework ROOT

ROOT può essere utilizzato in due modi, entrambi illustrati di seguito:

- Tramite **Macro C++** (file `.C`), che vengono compilate o interpretate nativamente.
- Tramite **PyROOT** (file `.py`), un *wrapper* che permette di usare le classi C++ di ROOT all'interno di uno script Python.

Oggetti fondamentali in ROOT Nella sintassi di ROOT, ogni oggetto inizia tipicamente per T:

- **TCanvas**: Dove vengono disegnati i grafici.
- **TH1F**: Istogramma monodimensionale (1D) con frequenze in formato *float*.
- **TGraphErrors**: Grafico a dispersione per coppie di dati (x, y) con rispettivi errori (σ_x, σ_y) .
- **TF1**: Funzione matematica 1D, utilizzata per eseguire i fit.

Fit Base: Istogramma e Gaussiana (Macro C++)

Questo esempio mostra come leggere una colonna di dati, riempire un istogramma TH1F e fittarlo con una funzione di Gauss. In ROOT, funzioni comuni come la Gaussiana sono già predefinite nella libreria matematica TMath. Di seguito è riportato il codice in C++ (`fit.C`):

Listing 1.3: Macro C++ per fit Gaussiano con ROOT

```
1  #include "TMath.h"
2  #include "TF1.h"
3  #include "TH1F.h"
4  #include "TCanvas.h"
5  #include <fstream>
6
7  // Definizione della funzione di fit
8  double fitfunc(double* x, double* p) {
9      double norm = p[0];
10     double mean = p[1];
11     double sigma = p[2];
12     return norm * TMath::Gaus(x[0], mean, sigma);
13 }
14
15 void fit() {
16     const int mpt = 9999;
17     float x1[mpt];
18     int npt = 0;
19
20     // 1. Lettura dati da file
```

```

21 std::ifstream in("dat.txt");
22 while (in.good()) {
23     in >> x1[npt];
24     if (!in.good()) break;
25     npt++;
26 }
27 in.close();
28
29 // 2. Creazione Canvas e Istogramma
30 TCanvas* c1 = new TCanvas("c1", "c1", 0, 0, 500, 500);
31 TH1F* h1 = new TH1F("h1", "h1", 100, -10.0, 25.0);
32
33 for (int i = 0; i < npt; i++) {
34     h1->Fill(x1[i]);
35 }
36
37 // 3. Definizione TF1 e Fit
38 TF1* f1 = new TF1("f1", fitfunc, -100.00, 100.0, 3);
39 f1->SetParameters(7.100, -5.100, 0.100);
40 f1->SetParNames("A", "B", "C");
41
42 gStyle->SetOptStat(kFALSE);
43 gStyle->SetOptFit(kTRUE); // Mostra il box con i risultati del fit
44
45 c1->cd();
46 h1->Fit("f1");
47 f1->SetLineColor(kRed);
48 f1->Draw("same"); // Disegna la funzione sopra l'istogramma
49 }

```

Fit Avanzato: TGraphErrors e Residui (PyROOT)

Nel caso di misure (x, y) con incertezze su entrambi gli assi, si utilizza la classe `TGraphErrors`. Questo script mostra l'implementazione in **PyROOT**.

Lo script esegue il fit dei dati, calcola gli errori efficaci, costruisce l'istogramma dei residui e verifica l'assenza di trend sistematici tramite un fit lineare (polinomio di grado 1: `pol1`):

Listing 1.4: Fit con Varianza Efficace e Residui tramite PyROOT

```

1 import ROOT
2 import array
3
4 # 1. Definizione della funzione (Esponenziale)
5 def fitfunc(x, p):
6     return p[0] * ROOT.TMath.Exp(-x[0] / p[1])
7
8 # Inizializzazione array tipizzati (necessari per interfacciarsi col C++ di
   ROOT)

```

```
9 x1, x2, x3, x4 = array.array('f'), array.array('f'), array.array('f'), array.  
    array('f')  
10  
11 # [ ... Lettura file datpy.txt omessa per brevità ... ]  
12  
13 npt = len(x1)  
14  
15 # 2. Creazione Graph e Fit Principale  
16 c1 = ROOT.TCanvas('c1', 'Fit Esponenziale', 0, 0, 500, 500)  
17 gr = ROOT.TGraphErrors(npt, x1, x2, x3, x4)  
18  
19 f1 = ROOT.TF1('f1', fitfunc, 0.00, 10.00, 2)  
20 f1.SetParameters(0.0, 1.0)  
21 f1.SetParNames('A', 'B')  
22  
23 ROOT.gStyle.SetOptFit(ROOT.kTRUE)  
24  
25 c1.cd()  
26 gr.Fit(f1) # ROOT si occupa della minimizzazione  
27 gr.SetLineColor(ROOT.kBlue)  
28 gr.Draw('AP') # A=Assi, P=Punti  
29 f1.SetLineColor(ROOT.kRed)  
30 f1.Draw('same')  
31  
32 # 3. Analisi dei Residui Normalizzati (con errore efficace calcolato da ROOT)  
33 x4b = array.array('f') # conterra' gli errori efficaci calcolati  
34  
35 for i in range(npt):  
36     # Varianza efficace usando f1.Derivative()  
37     var_eff = x4[i]**2 + (x3[i]*f1.Derivative(x1[i]))**2  
38     x4b.append(ROOT.TMath.Sqrt(var_eff))  
39  
40 h1 = ROOT.TH1F('h1', 'Distribuzione Residui', 20, -10.0, 10.0)  
41  
42 for i in range(npt):  
43     resid = (x2[i] - f1.Eval(x1[i])) / x4b[i]  
44     h1.Fill(resid)  
45  
46 # Fit dei residui con Gaussiana  
47 c2 = ROOT.TCanvas('c2', 'Residui', 600, 0, 500, 500)  
48 fgaus = ROOT.TF1('fgaus', 'gaus', -2.0, 2.0)  
49 fgaus.SetParameters(3.0, 0.0, 1.0)  
50 c2.cd()  
51 h1.Fit(fgaus)  
52  
53 # 4. Trend dei residui vs x  
54 y2 = array.array('f')  
55 for i in range(npt):  
56     y2.append(x2[i] - f1.Eval(x1[i]))  
57  
58 c3 = ROOT.TCanvas('c3', 'Trend Residui', 300, 300, 500, 500)
```

```
59 gr1 = ROOT.TGraphErrors(npt, x1, y2, 0, x4b) # X ha errore 0 qui per il plot
60 g1 = ROOT.TF1('g1', 'pol1', 0.00, 10.00) # Fit lineare
61
62 c3.cd()
63 gr1.Fit(g1)
64 gr1.Draw('AP')
65 g1.Draw('same')
66
67 # Salvataggio su file
68 c1.SaveAs('c1.png')
69 c2.SaveAs('c2.png')
70 c3.SaveAs('c3.png')
```